

NeuFlow v3

Queried optical flow with calibrated uncertainty
for region-directed perception

Shriman Raghav Srinivasan

MS Robotics, Northeastern University · Field Robotics Lab · August 2026

Abstract

Real-time optical flow networks emit a displacement for every pixel, at one fixed resolution, on every call, and retain no state between calls. Many consumers of flow, including registration, sparse tracking, visual odometry and survey mosaicking, need a few hundred correspondences at locations they select, and a platform working to a millisecond budget can afford neither the full field nor the recomputation. This report presents NeuFlow v3, which retains the NeuFlow v2 pipeline through its coarse flow stage and replaces only the final learned convex upsampler with an implicit decoder evaluated at arbitrary continuous coordinates. The front end is frozen and verified bit-identical across all 137 shared tensors after every run, so every measured difference is attributable to the decoder alone. The decoder samples four feature sources in a 3×3 window around a query, fuses them through a gated network, and emits a convex combination of ten candidate displacements drawn from the coarse field, bounding the prediction inside the locally supported motion range, together with a per-query error scale trained under a Laplace likelihood.

The resulting factorisation into a cached coarse pass and a queried decode changes the cost of N queries about one frame pair from $33.3N$ ms to $32.8 + 1.25N$ ms. A repeat query on a cached pair costs 1.25 ms against 33.3 ms, and an object appearing in a frame already being processed is answered from cache in 1.68 ms at 2.10 px, against 7.29 ms and 3.60 px for a fresh crop through the baseline, which is both slower and less accurate because cropping discards the context global matching requires. Sparse output reproduces the dense field to 0.00057 px, so sparse and dense decoding are one function evaluated on two point sets rather than two operating modes. Across 2,348,000 held-out queries the predicted error scale orders measured error monotonically over five equal-count bins, from 0.48 px to 7.10 px; abstaining on the least confident fifth reduces mean error over the accepted 80% to 1.06 px, a selection the baseline cannot make at any threshold because it emits flow alone.

The cost is quantified rather than avoided. Mean end-point error rises from 2.324 px to 2.384 px, 2.6%, from a model 13% smaller, and the like-for-like single-dataset comparison is 2.500 px. The penalty is concentrated in sub-pixel precision, 1.5 points of 1-pixel accuracy, rather than in large errors, and dense reconstruction is 11% slower, so the baseline remains the correct choice for consumers that take one dense field per frame. Because every driving result is measured on a domain v3 trained on and the baseline did not, an unconfounded cross-domain comparison is reported on aerial imagery neither model trained for, where the queried decoder records lower error at all seven crop settings tested, by consistent but small margins. Finally, region-directed inference for fast-moving platforms is characterised: processing 14% of the frame costs 0.034 px, and the context margin a cropped region requires is set by inter-frame displacement rather than by image size, a relation that collapses across a threefold difference in motion scale and can therefore be sized from platform speed and frame rate before any flow is computed.

1. Why queryable flow

Flow networks produce a dense map. Many consumers do not want one.

- Registration needs a few hundred correspondences at points it picks itself.

- Sparse tracking needs flow only at feature points.
- Survey mosaicking needs matches in overlap regions, often between pixels.

Computing 479,232 values to use 800 of them is the difference between real time and not on constrained hardware. A fixed-resolution design also cannot answer any query above its input sampling rate, which is what high-resolution registration needs.

The question this project asks: can the last stage be made queryable without losing accuracy or speed?

2. Related work

Flow started as a variational problem. Horn and Schunck [1] minimise a global energy pairing brightness constancy with a smoothness penalty on the field. Lucas and Kanade [2] solve brightness constancy in closed form over a window, assuming the flow is constant inside it. Both are local by construction, because the data term is a first-order expansion about zero displacement and holds only while motion is small relative to image structure. Brox et al. [3] add gradient constancy and a warping scheme that solves at reduced resolution first, which extends the usable range but still loses a small object that moves far enough to vanish from the coarsest level. Large displacement of small structures is the failure every learned method since has been measured against.

FlowNet [4] replaced the energy with a network trained end to end on synthetic data, and showed that an explicit correlation layer between the two frames helps more than letting a generic encoder discover matching on its own. It was well short of variational accuracy. FlowNet 2.0 [5] closed that gap by stacking several such networks with warping between them and a separate branch for small displacement, at the cost of a large model.

PWC-Net [6] and LiteFlowNet [7] put the classical pyramid back inside the network. Features are extracted at several scales, flow from the coarser level warps the second frame, and a cost volume is built over a bounded displacement range around that warp. Bounding the search keeps the volume small, which is what made these models compact enough to be practical. The cost is the cascade’s own weakness: an error made at a coarse level is never revisited. RAFT [8] removed the cascade by building one all-pairs correlation volume at $1/8$ resolution and reading from it repeatedly with a recurrent unit, refining a single estimate at fixed resolution. It set the accuracy standard the field still works against, and it is slow. GMFlow [9] treats correspondence as global matching instead, with cross-attention over the whole feature map producing flow directly and refinement reduced to a correction.

Real-time work asks what of this survives a millisecond budget. NeuFlow [10] keeps global matching but runs it at $1/16$ behind a shallow backbone, then refines with cheap recurrent updates. NeuFlow v2 [11] restructures the backbone and refinement and ends with a learned convex upsampler that predicts blending weights over a 3×3 neighbourhood of coarse flow. That upsampler is the stage v3 replaces, and everything above it is what v3 inherits frozen.

Implicit representations arrived from elsewhere. NeRF [12] showed a coordinate-conditioned MLP can hold a scene and be evaluated anywhere in it, and SIREN [13] showed the activation function decides how much high-frequency detail such a network can hold at all. LIIF [14] made the idea discrete to continuous for images: a local latent code plus a relative coordinate, decoded by one shared MLP, so a single trained model can be sampled at any resolution. AnyFlow [15] brought arbitrary-scale decoding to flow and predicts convex weights over candidate displacements, which is where v3’s ten-candidate head comes from. InfiniDepth [16] does the continuous version for depth and contributes the gated hierarchical fusion of multi-scale features that v3’s decoder uses.

Confidence has its own line. Kendall and Gal [17] separate aleatoric from epistemic uncertainty

and train a heteroscedastic likelihood whose predicted scale is supervised only by the error it explains. Ilg et al. [18] applied this to flow directly, comparing multi-hypothesis networks against a single predictive distribution and showing the estimate is usable rather than decorative. PDC-Net [19] predicts a mixture density per correspondence and uses it to decide which matches to trust, which is the same use v3 makes of its per-query scale.

Every method above emits a dense field at one fixed resolution, chosen when the network is built. The queryable ones are queryable in scale, not in coordinate, and the implicit decoders that are queryable in coordinate have not been asked to run inside a network with a millisecond budget. So whether arbitrary-coordinate querying is affordable at speed is open. That is what this report measures.

3. Method

3.1 What is inherited

NeuFlow v2 [11] runs a shallow CNN backbone at 1/8 and 1/16 resolution, cross-attention and global matching at 1/16 for an initial flow, recurrent refinement (one iteration at 1/16, eight at 1/8), then a learned convex upsampler to full resolution.

v3 keeps all of that up to the 1/8 coarse flow, with every learned weight frozen and every batch-normalisation running statistic held at v2’s value. I verify this per checkpoint: all 137 shared tensors are bit-identical to v2, so the comparison below isolates the decoder and nothing else.

3.2 The decoder

Two phases.

Phase 1, once per frame pair, 32.8 ms. The frozen pipeline produces the coarse flow and feature maps. These are cached.

Phase 2, once per query batch, 1.25 ms for up to 2,048 points. For a coordinate (x, y) the decoder samples 3×3 windows from four sources (1/8 context, 1/8 features, 1/16 features, flow-warped frame-1 features), fuses them in a gated MLP, and predicts weights over ten candidates: the nine coarse-flow values around the query plus a bilinear sample.

Output is a convex combination of those candidates. That bounds the prediction inside the locally supported motion range, so the decoder cannot invent displacement its inputs do not support. The head is initialised so the softmax sits on the bilinear candidate, which makes the untrained decoder exactly equivalent to bilinear upsampling (measured: 0.011 px maximum deviation). Training can only move away from that by learning.

3.3 Uncertainty head

An optional eleventh output predicts a per-query error scale b under a Laplace likelihood. It costs no measurable inference time and attaches a confidence value to every correspondence returned.

3.4 Interface

```
state = model.infer_coarse_state(img0, img1)           # once per pair
flow  = model.decode_queries(state, query_coords=q)    # [B,N,2] -> [B,N,2]
flow  = model.decode_queries(state, target_h=H, target_w=W)
flow, b = model.decode_queries(state, query_coords=q, return_uncertainty=True)
```

Coordinates are continuous. (312.7, 188.2) is as valid as (312, 188). Sparse queries reproduce the dense field exactly at matching coordinates, to 0.00057 px.

4. Setup

Evaluation. VKITTI2 Scene18 and Scene20: 1,174 pairs, 460,573,660 valid pixels, per-pixel metrics. Both scenes are excluded from every training set. A guard in the loader enforces it and I verify no overlap at pair and frame level before each run.

Controls. Every training run is generated from one template, so any two differ in exactly one variable. All use seed 1234, batch 16, learning rate 2×10^{-4} on a OneCycle schedule, $\gamma = 0.8$, 4,096 supervision queries per image, refinement of $1 \times s16$ and $8 \times s8$ matching evaluation, and 100,000 steps.

Pre-flight. A ten-check suite runs on CPU and GPU before any job: batch-normalisation frozen, only decoder parameters trainable, zero-initialised decoder equals bilinear, sparse equals dense, uncertainty present in both decode paths, stride-2 fidelity, seed reproducibility.

Hardware. RTX 4060 laptop, fp16, 384×1248 input unless stated.

5. Results

5.1 Accuracy

Model	Training data	EPE (px)	1px (%)	3px (%)	Params
NeuFlow v2	FlyingThings	2.324	77.63	89.80	9.03 M
NeuFlow v3	FlyingChairs	2.500	72.81	87.88	7.83 M
NeuFlow v3	+ VKITTI2	2.398	75.74	88.94	7.83 M
NeuFlow v3	+ MPI-Sintel	2.392	75.83	88.98	7.83 M
NeuFlow v3	+ uncertainty head	2.384	76.13	89.02	7.83 M

Every v3 configuration sits above v2. The best, at 2.384 px, is 2.6% worse on mean error and 1.5 points worse on 1-pixel accuracy. The like-for-like comparison is worse still: trained on FlyingChairs alone, mirroring v2’s own training, v3 reaches 2.500 px, which is 7.6% behind.

So the implicit decoder is less accurate than v2’s convex upsampler. The reason is identified in Section 8 and is not a training artefact: the decoder’s finest input sits at $1/8$ resolution while v2’s upsampler reads the full-resolution frame.

The ordering is monotonic. Each addition of data helps, and the uncertainty head is best of the four, though the last three differ by less than 0.02 px and I would not read much into their ranking from a single seed.

One qualification applies to this table and to every driving result that follows. v3’s training mix includes other VKITTI2 scenes while v2’s does not, so the comparison is biased in v3’s favour on this domain. The accuracy gap above is therefore a lower bound on the true cost of the decoder, and the only unconfounded comparison in this report is the aerial one in Section 5.5, on a domain neither model trained for.

5.2 Compute

Mode	Latency	Note
NeuFlow v2, full frame	33.3 ms	its only mode
v3 sparse, first query on a new pair	34.1 ms	level with v2
v3 sparse, repeat query on a cached pair	1.25 ms	27× cheaper
v3 dense, stride 2	37.1 ms	not the intended mode

The sparse path is a 32.8 ms coarse pass inherited from v2 plus a 1.25 ms decode. Global matching needs whole-image context, so the coarse pass cannot be skipped. It is 96% of a first query, which

is why a first query costs what a v2 frame costs.

State reuse is the property that matters. v2 keeps nothing between calls, so asking a second question about the same frame means recomputing everything. v3 answers from cached state in 1.25 ms. For anything that queries a frame more than once (iterative registration, RANSAC refinement, interactive selection, multi-hypothesis tracking) that is a structural difference, not a margin.

Decode latency is 1.254 ms at $N = 800$ and 1.285 ms at $N = 2,048$. The decode is bound by kernel launch overhead, not compute, so 2,048 points cost the same as 800.

5.3 Calibrated uncertainty

Predicted scale b	Actual mean error
0.01 to 0.10	0.480 px
0.10 to 0.19	0.896 px
0.19 to 0.39	1.018 px
0.39 to 1.22	1.837 px
above 1.22	7.100 px

Measured over 2,348,000 queries. Actual error rises monotonically across all five bins, a $15\times$ span, Pearson $r = 0.318$. The correlation is not strong but the ordering is reliable and directly usable: weight correspondences in RANSAC, drop unreliable ones, or send more queries where the model is unsure. The ordering can be trusted; the per-query magnitude cannot, and I do not claim it. v2 outputs flow only and has nothing comparable.

5.4 Abstention: what the ordering is worth

Because the model can rank its own answers it can decline to answer some of them. Sorting the same 2,348,000 queries by predicted confidence and answering only the most confident fraction gives an accuracy-coverage curve.

Coverage	EPE of the accepted set	v2
20%	0.480 px	no selection possible
80%	1.058 px	no selection possible
100%	2.266 px	2.324 px

Discarding the least confident fifth reduces error over the remaining 80% of the frame to 1.058 px. The claim I make without qualification is the internal one: across 2.35 M of my own queries, error falls monotonically as predicted confidence rises, and the spread from the most to the least confident group is more than fourfold. That is measured on my own outputs and depends on no comparison.

The comparison against v2 is the weaker framing and I would rather state its weakness than have it extracted. The accepted 80% is the easier 80%, and v2 would also score better on that subset than on the full frame. What survives regardless is the capability: there is no threshold that can be set on a model emitting flow alone to produce this curve at all. v2 has one operating point, structurally. If a query is rejected the consumer asks elsewhere or falls back on its previous estimate, and asking again costs about a millisecond.

5.5 Aerial domain: the one unconfounded comparison

Every driving result above is measured on a domain v3 trained on and v2 did not. TartanAir is not: neither model has trained on aerial imagery, which makes it the only clean head-to-head in this study. The same 40 held-out pairs are scored at the full frame and at six crop margins.

Crop margin	NeuFlow v2	NeuFlow v3
Full frame	0.781	0.777
0 px	1.205	1.176
8 px	0.901	0.881
16 px	0.872	0.865
24 px	0.879	0.854
32 px	0.834	0.814
64 px	0.793	0.778

v3 records the lower mean end-point error at every one of the seven settings. The differences are small and worth sizing out loud: they run from 0.004 to 0.029 px, on 40 pairs and a single seed, so no individual difference is meaningful. What makes the table worth showing is that the direction repeats across seven independent settings.

My explanation is a hypothesis rather than a finding: v3 trained on three datasets where v2 trained on one, and a bounded decoder degrades more gracefully on unfamiliar input because it cannot leave the locally supported motion range whatever the network predicts. It is also not uniformly better. On large-motion failures v3 is worse, 7.2% against 6.5%. Forty pairs on one seed is a consistent signal, not a reversal of the accuracy result in Section 5.1.

5.6 Interactive tool

A PyQt application loads a video, steps frame by frame, and computes flow for a region the user drags out. Two modes: exact decoding inside the box against a full-frame coarse pass, or a cropped mode running the whole pipeline on the selection so cost scales with the area requested. Both print their latency breakdown live.

6. Flowing only part of the frame

A fast platform cannot afford full-frame flow every frame, so it flows only the region that matters. Three questions follow: what cropping to a region costs, when it pays, and what happens when a new object appears in a frame already being processed.

6.1 What a region costs

Region processed	Area	Latency	Speedup	EPE in region
Full frame	100%	33.3 ms	1.0×	0.657 px
Region, no margin	7.9%	7.8 ms	4.3×	1.089 px
Region + 32 px margin	13.8%	7.6 ms	4.4×	0.691 px
Region + 64 px margin	20.1%	8.1 ms	4.1×	0.667 px
Region + 128 px margin	34.2%	9.9 ms	3.4×	0.655 px

Keeping full resolution and processing 14% of the frame costs 0.034 px and runs 4.4× faster. Beyond a 32 px margin nothing improves, so the extra area is wasted.

Cropping applies to any flow network, v2 included. It is a platform technique, not a property of the queryable decoder.

6.2 The margin rule

A crop removes the context global matching uses to find large displacement. With no margin, error rises 0.432 px and 24% of large-motion pixels fail outright. A 32 px margin on 26.6 px mean motion costs 0.034 px and is the knee; beyond it nothing improves. The rule is **margin** $\approx$ **expected inter-frame motion** $\approx$ **speed / frame rate**, so a faster platform needs a larger

margin and saves proportionally less.

6.3 The rule across two motion scales

Tested on aerial sequences, which move very differently from driving: 9.26 px mean in-region displacement against 26.6 px.

Domain	Mean motion	Margin needed	Penalty there
Driving	26.6 px	~32 px	+0.034 px
Aerial	9.26 px	~8 px	+0.120 px

Both begin at the same 0.43 px penalty with no margin, and both have shed most of it once the margin reaches one frame of motion. The requirement is therefore set by displacement rather than by image size, and can be sized from platform speed and frame rate before any flow is computed. The agreement is close up to that point; the residual beyond it differs by scene, so this predicts the scale of the margin rather than the exact curve.

6.4 A new object mid-frame

Policy	Cost of the new object	EPE on it
v3, sparse queries (800 pts)	1.68 ms	2.10 px
v3, dense ROI	4.36 ms	1.77 px
v2, new crop	7.29 ms	3.60 px
v3, new crop	8.23 ms	3.61 px

Because the coarse pass is cached, the decoder answers a newly appeared object for 1.68 ms. A cropped pipeline must run an entire new pass and is markedly less accurate doing so, having lost the global context. NeuFlow v2 keeps no state between calls and has nothing to answer from. This is the one result in this section specific to the architecture rather than to cropping: answering a question that was not anticipated when the frame arrived.

Two disjoint crops cost more than one full frame, so the policy rule is to crop once or not at all.

7. What v3 is for

The contribution is an operating point, not a leaderboard position. v3 trades 2.6% of mean accuracy and 1.5 points of 1-pixel accuracy for three properties the fixed-resolution design cannot express, from a model 13% smaller:

1. Flow at any continuous coordinate, with sparse output identical to dense.
2. Repeat access at 1.25 ms per query batch against a cached frame, versus a full 33.3 ms recomputation.
3. A calibrated error estimate on every correspondence.

If an application consumes one full dense flow field per frame and nothing else, v2 is the better choice on every axis: more accurate, and 11% faster in that mode. I would say so plainly. v3 earns its place when the consumer picks its own query points, revisits a frame, needs positions between pixels, or wants a confidence value, and when 2.6% of mean accuracy is an acceptable price for those.

8. Limitations

Accuracy. The decoder is less accurate than the upsampler it replaces, by 2.6% on mean error and 1.5 points of 1-pixel accuracy at best, 7.6% on the like-for-like comparison. I know the cause: the decoder’s finest input is at $1/8$ resolution, so the evidence it sees barely changes inside an 8×8 cell, while v2’s upsampler reads the full-resolution frame directly. Three independent experiments converge on that same mechanism. A Fourier positional encoding of the sub-cell offset returned null, which rules out a missing positional signal. Sub-pixel querying gains very little. Querying below the input sampling rate matches bilinear interpolation exactly. That convergence is what makes the resolution ceiling a diagnosis rather than a suspicion. It is diagnosed and unfixed in this work.

The same limitation means sub-pixel querying is currently closer to interpolation than to genuine sub-pixel inference: queries within one 8×8 cell see nearly identical evidence. The interface is exact and the mechanism is real, but the extra information it can return between pixels is small until the decoder sees full-resolution features.

No embedded measurement. Every latency figure is from a laptop RTX 4060. The edge claim is a design target, not a result. The region speedup is hardware dependent for the same reason: a small region becomes launch-overhead bound on a larger accelerator. The accuracy cost of a region is not hardware dependent.

One primary evaluation domain. VKITTI2 is synthetic driving footage, and it says little on its own about field or survey imagery. The 40 aerial TartanAir pairs answer that only partly.

Statistical resolution. One seed per configuration, with checkpoint-to-checkpoint variation up to 0.038 px. Differences below about 0.05 px are not resolved.

9. Next

Ordered by what each item settles, not by how much work it takes.

1. **Embedded measurement on a Jetson.** Converts the edge claim into a result, or refutes it. Nothing else in this work depends on the measurement platform the way this does.
2. **A second seed per configuration,** one training run each. Three of the four configurations currently sit inside the noise floor and cannot be ranked. A second seed is what would allow ranking them honestly.
3. **Evaluation above the input sampling rate.** Query at $2 \times$ resolution against native 4K ground truth. The script exists and runs today. This is the one test v2 structurally cannot take, because the question cannot be posed to it.
4. **Tiled refinement.** Refinement is 60% of runtime and all of its operations are local, so it should run on region tiles with a halo, and the measured halo decay indicates no retraining is needed. Estimated $2 \times$. This is not a v3 contribution: it would benefit v2 equally.
5. **Registration on field and survey imagery,** which tests the intended application and addresses the single-domain limitation directly.

References

- [1] B. K. P. Horn and B. G. Schunck, “Determining optical flow,” *Artificial Intelligence*, vol. 17, no. 1–3, pp. 185–203, 1981.
- [2] B. D. Lucas and T. Kanade, “An iterative image registration technique with an application to stereo vision,” in *Proc. Int. Joint Conf. Artificial Intelligence (IJCAI)*, 1981, pp. 674–679.
- [3] T. Brox, A. Bruhn, N. Papenberger, and J. Weickert, “High accuracy optical flow estimation based on a theory for warping,” in *Proc. European Conf. Computer Vision (ECCV)*, 2004.

- [4] A. Dosovitskiy, P. Fischer, E. Ilg, P. Häusser, C. Hazirbas, V. Golkov, P. van der Smagt, D. Cremers, and T. Brox, “FlowNet: Learning optical flow with convolutional networks,” in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2015.
- [5] E. Ilg, N. Mayer, T. Saikia, M. Keuper, A. Dosovitskiy, and T. Brox, “FlowNet 2.0: Evolution of optical flow estimation with deep networks,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [6] D. Sun, X. Yang, M.-Y. Liu, and J. Kautz, “PWC-Net: CNNs for optical flow using pyramid, warping, and cost volume,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [7] T.-W. Hui, X. Tang, and C. C. Loy, “LiteFlowNet: A lightweight convolutional neural network for optical flow estimation,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [8] Z. Teed and J. Deng, “RAFT: Recurrent all-pairs field transforms for optical flow,” in *Proc. European Conf. Computer Vision (ECCV)*, 2020.
- [9] H. Xu, J. Zhang, J. Cai, H. Rezatofighi, and D. Tao, “GMFlow: Learning optical flow via global matching,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [10] Z. Zhang, H. Jiang, and H. Singh, “NeuFlow: Real-time, high-accuracy optical flow estimation on robots using edge devices,” *arXiv:2403.10425*, 2024.
- [11] Z. Zhang, A. Gupta, H. Jiang, and H. Singh, “NeuFlow v2: High-efficiency optical flow estimation on edge devices,” *arXiv:2408.10161*, 2024.
- [12] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, “NeRF: Representing scenes as neural radiance fields for view synthesis,” in *Proc. European Conf. Computer Vision (ECCV)*, 2020.
- [13] V. Sitzmann, J. N. P. Martel, A. W. Bergman, D. B. Lindell, and G. Wetzstein, “Implicit neural representations with periodic activation functions,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [14] Y. Chen, S. Liu, and X. Wang, “Learning continuous image representation with local implicit image function,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [15] H. Jung, Z. Hui, L. Luo, H. Yang, F. Liu, S. Yoo, R. Ranjan, and D. Demandolx, “AnyFlow: Arbitrary scale optical flow with implicit neural representation,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [16] H. Yu, H. Lin, J. Wang, J. Li, Y. Wang, X. Zhang, Y. Wang, X. Zhou, R. Hu, and S. Peng, “InfiniDepth: Arbitrary-resolution and fine-grained depth estimation with neural implicit fields,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2026. arXiv:2601.03252.
- [17] A. Kendall and Y. Gal, “What uncertainties do we need in Bayesian deep learning for computer vision?” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [18] E. Ilg, O. Cicek, S. Galesso, A. Klein, O. Makansi, F. Hutter, and T. Brox, “Uncertainty estimates and multi-hypotheses networks for optical flow,” in *Proc. European Conf. Computer Vision (ECCV)*, 2018.
- [19] P. Truong, M. Danelljan, L. Van Gool, and R. Timofte, “Learning accurate dense correspondences and when to trust them,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2021.

Code, training configurations and the full development record: github.com/shrirag10/Neuflowv3. Every number here is reproducible from `scripts/eval_all_runs.py`, `scripts/benchmark_sparse.py`, `scripts/eval_calibration.py`, `scripts/eval_roi_crop.py` and `scripts/bench_scenarios.py`. Full parameter provenance is in `docs/base_parameters.md`.